International Black Sea University
Faculty of Computer Science / Engineering

A.L.F.R.E.D.

A Local-First, Two-Tier Routing Middleware
for On-Device Intent Execution

Bachelor Thesis

Author: David Zoidze
Supervisor: Giorgi Merabishvili

Tbilisi, 2026

Abstract

On-device assistants must act on a user's behalf—set an alarm, toggle a setting, place a call—without sending the request to the cloud. This thesis asks how much language model such *action* actually requires. We present A.L.F.R.E.D., a local-first middleware that splits every request between a tiny on-device *reflexer* (a 2B model that fills a distilled command pattern in a single forward pass) and a heavier *thinker* (an agent loop) reserved for the long tail. Across purpose-built benchmarks scored by an end-state oracle, four results establish the central claim. First, on binding the 2B reflexer with a distilled pattern matches a 17× larger free-form thinker (both 100%), while the same 2B without the pattern reaches only 30%—distillation, not scale, produces reliable execution. Second, the abstraction's benefit shifts with the interface: on a clean settings API it is an efficiency win (+65 points at two commands instead of nine), and on a realistic Android API it becomes an accuracy win. Third, and most importantly, even a 35B reasoning model fails the device's undiscoverable encoding conventions; model scale buys *discovery* (which key) but not *convention* (what the value means), which only a distilled pattern supplies. Fourth, that pattern can be minted automatically by a teacher that never sees the tasks, lifting the 2B reflexer from 25% to 76–80%—and the choice of teacher propagates directly to the student (a cloud teacher's patterns score 80% versus a local teacher's 60%). The remaining open limit is shown to be routing, not execution. The thesis frames these findings as the *executioner paradigm*: distill device conventions once with a strong teacher, and apply them cheaply forever with a small on-device reflexer.

Keywords: on-device language models, knowledge distillation, agent routing, tool use, intent execution, privacy.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context

A growing share of what people ask a phone to do is not a question but an *action*: set an alarm, dim the screen, start a timer, turn on Do Not Disturb, call a contact. These requests are short, frequent, and repetitive, and they share a property that the conversational use of assistants does not—they have a single correct outcome. Either the alarm is set for 7:00 or it is not.

How these actions are served is in the middle of an industry shift. The dominant pattern routes a request to a large cloud model; the emerging one keeps as much as possible *on the device*. Both Google (Gemini Nano through Android AICore) and Apple (Apple Intelligence, with Private Cloud Compute for overflow) now run a small model locally and escalate only when they must. The motivation is threefold: *privacy*, because an action request often contains personal context that should never leave the device; *cost and latency*, because a local model answers in milliseconds without a network round-trip or a per-call bill; and *availability*, because the device should still work offline. The open engineering question this shift raises is the subject of this thesis: how much language model does reliable on-device *action* actually require?

1.2 The problem

The difficulty is a squeeze from both ends. A model small enough to run comfortably on a phone is, on its own, an unreliable executor: asked to carry out a device action by free-form reasoning, it either fails to act at all or invents a command from its pretraining priors instead of the one the device expects (Chapter 5 measures both failures). A model large enough to reason reliably does not fit on the device, answers an order of magnitude more slowly, costs money per call, and—as this thesis shows—is *still* wrong on the device-specific conventions that no amount of reasoning can recover.

So neither obvious option is satisfactory. Shipping a big model defeats the point of going on-device; shipping a small free-form model is unreliable. The interesting design space is between them: a tiny model that does not have to be clever, paired with something that supplies the knowledge it lacks.

1.3 The gap

Two gaps in the existing picture motivate the work. First, the literature treats "the model is too small" as a single problem, when on a realistic device API it is really two. One is *discovery*—finding which setting, namespace, or key a request maps to—which is a search problem that a larger model solves by exploring the API. The other is *convention*—knowing that "total silence" is encoded as the integer `2`, or that "largest font" is the string `1.30`—which is not discoverable from the API at all, because the meaning lives in a device's tribal knowledge rather than in any queryable surface. These two difficulties respond to completely different remedies, and conflating them hides the most useful result in the thesis (Figure 5.3).

Second, the standard tool for compressing a capable model into a deployable one is knowledge distillation into a smaller network's weights. That approach yields an opaque student that must be retrained to extend. For an on-device system that skill authors must extend and users must trust, a different distillation target is attractive: compressing the teacher's knowledge into an *inspectable, plug-and-play symbolic artifact*. Whether such symbolic distillation is sufficient to make a tiny model a reliable executor has not, to our knowledge, been measured end-to-end.

1.4 Thesis statement and research questions

This thesis advances and defends a single claim:

> On the repeatable, deterministic *head* of on-device tasks, **distillation substitutes for scale**: a 2-billion-parameter executor handed a distilled symbolic pattern matches a 17× larger free-form agent in binding accuracy, at one on-device forward pass, because the bottleneck is not reasoning but device knowledge the API does not expose. The remaining open limit is *routing*, not execution.

The claim is decomposed into five research questions, each answered in Chapter 5:

RQ1 (scale vs. distillation)
How much model does reliable binding (intent → correct command) require, and can a distilled pattern close the gap a small model has?

RQ2 (the spectrum)
Does the value of the reflexer change with API difficulty—when is it an efficiency win and when an accuracy win?

RQ3 (the limit of scale)
Are there task difficulties that model scale cannot solve but a distilled pattern can?

RQ4 (teacher quality)
In teacher→student distillation, how much does the choice of teacher (cloud vs. local) propagate to the on-device student?

RQ5 (coverage and routing)
What fraction of real on-device use-cases does the approach address, and where is the end-to-end bottleneck?

1.5 Contributions

1. **A two-tier local-first architecture** (A.L.F.R.E.D.) and the *executioner paradigm*: a small reflexer treated as a constrained slot-filler, not a generator, with a heavier thinker reserved for the tail (Chapter 3).

2. **Symbolic distillation**: knowledge distillation into a plug-and-play JSON pattern rather than student weights, with the teacher→student loop measured end-to-end (Sections 5.6).

3. **The discovery-vs-convention distinction**: two kinds of task difficulty, separated by what model scale can and cannot fix, demonstrated on a realistic device API (Section 5.5).

4. **Two end-state-oracle benchmarks** (SET-1 clean, SET-2 Android-realistic) and a standardized binding-only harness, with an integrity protocol that forbids fabricated numbers and benchmark leakage into distilled artifacts (Chapter 4).

5. **A coverage denominator** anchored to real Pixel and Apple on-device use-cases, and an honest localization of the open bottleneck at routing rather than execution (Sections 5.7–5.8).

1.6 Thesis structure

Chapter 3 presents A.L.F.R.E.D. as a system—the two tiers, the pattern artifact, the skill model, and the distillation step that connects them. Chapter 4 specifies how the system is evaluated: the binding-only methodology, the end-state oracle, the benchmarks, and the

integrity rules. Chapter 5 is the core, establishing the five results above in turn. Chapter 6 draws out the implications for design and productization and states the threats to validity. Chapter 7 summarizes the contributions and outlines the next steps—device grounding, deterministic value transforms, and the routing work that the evaluation identifies as the remaining limit.

Chapter 2

Background and Related Work

A.L.F.R.E.D. sits at the intersection of four lines of work: small on-device language models, language-model tool use and agents, model routing and cascading, and the distillation of agent experience into reusable skills. This chapter reviews each in turn and ends by stating precisely what gap the thesis fills. A conceptual thread runs underneath the whole design and is worth naming first. The two-tier split—a fast, automatic path and a slow, deliberate one—is the computational analogue of two old ideas: the reflex arc, a stimulus-driven motor response routed without higher deliberation [1], and the dual-process account of cognition, in which a fast automatic system handles the routine and a slow effortful system handles the novel [2]. The reflexer is the fast path; the thinker is the slow one.

2.1 Small on-device language models

The premise that a small model is sufficient for most agentic work is now argued directly. Belcak et al. [3] contend that small language models, not frontier models, are the right substrate for agentic systems, because agent workloads are dominated by narrow, repetitive sub-tasks that a small specialized model can serve at a fraction of the cost. A.L.F.R.E.D. is a concrete instantiation of that thesis for the on-device action setting, and supplies the mechanism—distilled patterns—that the position paper leaves open.

On the systems side, TinyAgent [4] demonstrates reliable function calling at the edge with small models, using fine-tuning and a constrained decoder to keep a small model on the rails of a fixed tool schema. A.L.F.R.E.D. shares the goal of reliable small-model tool use on-device but takes a different route to reliability: rather than fine-tuning the model to a schema, it hands the model a distilled pattern at inference time, keeping the model itself untouched and the capability set extensible by adding patterns. Industry on-device stacks—Gemini Nano through Android AICore, and Apple Intelligence with Private Cloud Compute for overflow—make the same architectural bet of a local small model with selective escalation; A.L.F.R.E.D. is a research-grade realization of that pattern whose tiers

can be measured in isolation.

2.2 Tool use and agent reasoning

The thinker tier implements the now-standard reasoning-and-acting loop introduced by ReAct [5], in which a model interleaves reasoning steps with tool calls and observations until a task is complete. The broader line on teaching models to use tools includes Toolformer [6], which shows a model can learn to call APIs from self-supervised data, and Gorilla [7], which connects a model to massive real API collections and confronts the problem of selecting the correct call from many candidates. These establish that capable models can use tools; the question this thesis asks is the inverse one, namely how little model is needed once the *which-tool* and *which-shape* decisions have been settled in advance by a distilled pattern. Where ReAct spends model capability at run time to decide how to act, the reflexer spends it once, offline, and replays the decision.

2.3 Routing and cascading

Deciding which model or path should serve a request is the routing problem. RouteLLM [8] learns to route between a strong and a weak model from human preference data, optimizing the cost-quality trade-off; Dekoninck et al. [9] give a unified treatment of routing and cascading and frame multi-tier systems as structured cascades, which is exactly how A.L.F.R.E.D.'s router-then-reflexer-or-thinker path should be understood. The difference in emphasis matters for this thesis. The routing literature largely treats the downstream models as black boxes and optimizes the *decision*; A.L.F.R.E.D. instead invests in making the cheap downstream path itself reliable through distillation, and then reports honestly that the *router* remains the binding constraint on end-to-end accuracy (Section 5.8). The two concerns are complementary: a better router and a more reliable reflexer multiply rather than substitute.

2.4 Skill memory and procedural distillation

The closest neighbours come from work that turns agent experience into reusable skills. Voyager [10] introduced the skill-library framing, in which an agent accumulates a growing library of executable skills and composes them; Memp [11] studies procedural memory directly, abstracting successful trajectories into reusable scripts. A.L.F.R.E.D.'s patterns can be read as the deterministic compilation of exactly such procedural abstractions: where Memp keeps scripts the agent re-interprets, A.L.F.R.E.D. freezes them into a template whose only run-time freedom is slot-filling.

The Memento line is the nearest published relative. Memento [12] adapts an agent without updating model weights by retrieving from a growing case bank—non-parametric personalization, the same high-level strategy A.L.F.R.E.D. uses with patterns. Its successor, Memento-Skills [13], adds a *learned* skill-router and reflective read-write learning over free-form markdown skills. A.L.F.R.E.D. contrasts on two axes: its skills are *compiled, deterministic* patterns rather than free-form documents, and (as the evaluation will argue) the production end is best served by a constrained executor whose command shape is fixed, trading the generality of a reflective skill for the reliability of a frozen one. The opposite design point—updating model weights to personalize an agent—is represented by reinforcement-learning approaches such as OpenClaw-RL [14]; this is the path A.L.F.R.E.D. explicitly does not take, because weight updates sacrifice the plug-and-play, inspectable extensibility that a symbolic pattern provides.

A note on naming is needed to avoid confusion. Our *reflexer* is unrelated to Reflexion [15], which uses verbal self-feedback as a form of reinforcement to improve an agent across attempts. Reflexion adds deliberation; the reflexer removes it. The shared root word denotes opposite design intents.

2.5 Knowledge distillation, classical and symbolic

The mechanism that makes the cheap tier work is distillation, and its lineage is explicit. Classical knowledge distillation compresses a large teacher network into the *weights* of a smaller student, transferring the teacher's soft behaviour into a deployable model [16]. A.L.F.R.E.D. distills along the same teacher-to-student axis but changes the target: the teacher's knowledge of device conventions is compressed into an inspectable *symbolic* artifact—a pattern—rather than into student parameters. The trade is deliberate. Weight distillation yields a general but opaque student that must be retrained to extend; symbolic distillation yields a student that is plug-and-play, auditable, and composable, at the cost of covering only distilled intents. This positions the contribution against both the distillation literature (same teacher-student idea, different artifact) and the programmatic-prompt line such as DSPy [17], which compiles declarative model calls into optimized pipelines; A.L.F.R.E.D. similarly compiles intents into fixed command shapes, but targets on-device execution rather than prompt-program optimization.

2.6 Evaluation of agents

Agent evaluation has converged on task suites that score completion of realistic multi-step goals—AgentBench [18] across diverse environments, and AssistantBench [19] for realistic, time-consuming web tasks. A.L.F.R.E.D.'s benchmarks share their realism goal but differ in instrument: rather than scoring trajectory completion, the settings benchmarks score

the *end state* of a mutable backend with a no-collateral constraint (Chapter 4), which isolates the specific failure of emitting a plausible command that produces the wrong device state. The skill interface itself follows the emerging `SKILL.md` convention for equipping agents with packaged capabilities [20], with which A.L.F.R.E.D. is structurally compatible.

2.7 The gap

Across these lines, three gaps remain that this thesis addresses. First, the small-model and routing literatures optimize the *decision* of which model to use but rarely quantify how reliable the cheap path can be made, or by what mechanism; A.L.F.R.E.D. measures exactly this and finds distillation, not scale, to be the lever (Chapter 5). Second, the skill-memory line keeps skills as re-interpreted scripts or free-form documents; the consequence of *freezing* them into deterministic templates—higher reliability at the cost of generality—has not been measured against a strong free-form baseline on a realistic device API. Third, and most specific, no prior work separates the two difficulties of device action—*discovery* (which key) and *convention* (what the value means)—and shows that model scale fixes the first but not the second, while a distilled pattern fixes both. That separation, and the teacher-quality result that follows from it, is the contribution the remaining chapters defend.

Chapter 3

System Design

This chapter describes A.L.F.R.E.D. as a system: the two execution tiers, the artifacts that connect them, and the design principles that hold the whole thing together. The goal is to fix the vocabulary that the methodology and evaluation chapters reuse, and to make one architectural claim concrete before any number is reported: *most on-device requests do not need a model to reason; they need a model to fill in a blank.*

3.1 Overview: two tiers, one router

A.L.F.R.E.D. sits between a user's natural-language request and the device's capabilities. Every request enters a **router**, which makes a single decision: can this request be served by a known, repeatable *pattern*, or does it require open-ended problem solving? The first case is dispatched to the **reflexer**; the second to the **thinker**.

The split is an economic bet, not a capability ranking. The reflexer is cheap, fast, and runs comfortably on the device; it is also narrow, because it can only serve intents for which a pattern exists. The thinker is general but expensive: it runs an agent loop, costs an order of magnitude more tokens and latency per request, and—at the sizes that reason well—does not fit on the device at all. Neither tier is sufficient alone. The contribution of the architecture is the *division of labour*: route the high-frequency, repeatable head to the reflexer and reserve the thinker for the tail. Chapter 5 quantifies exactly where that bet pays.

3.2 The reflexer: an executioner, not a generator

The reflexer is a small language model (in this work, `qwen-3.5-2b`) whose job is deliberately impoverished. It does not decide *what* to do or *how* to do it. Given a request that the router has already matched to a pattern, the reflexer extracts the variable parts of the request—the *slots*—and substitutes them into a fixed command template supplied

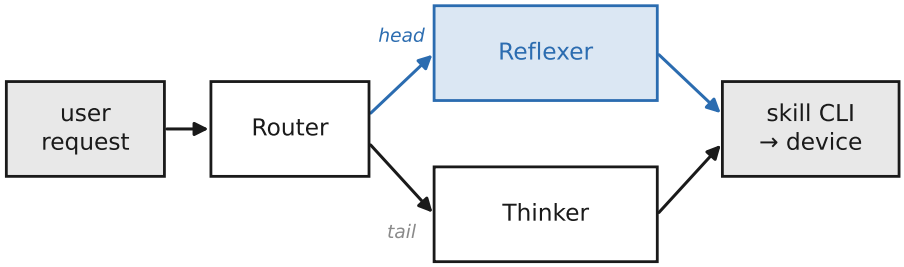

Figure 3.1: The two-tier architecture. The router classifies each request; the reflexer handles the deterministic head with a distilled pattern, the thinker handles the long tail with a full agent loop. Both emit commands to the same skill CLIs.

by that pattern. It then emits exactly one shell command. This is why we call it an *executioner* rather than a generator: it carries out a decision that was made once, offline, by something else.

This framing is the central design choice of the thesis, and it separates A.L.F.R.E.D. from the on-device assistant models shipped by Google and Apple. Those models are asked to be creative within a constrained surface; they generate. The reflexer is asked for none of that. It performs slot extraction and template binding, and nothing more. The benefit is reliability: the space of things the reflexer can get wrong shrinks to the space of slot values, because the command shape itself is not the model's to invent. The cost is generality, paid up front as the requirement that a pattern must exist (Section 3.6).

Concretely, the runtime is the short pipeline in Figure 3.2. The matched pattern supplies a *command shape* (the `args_template` with its slots rendered as typed placeholders) and a list of slot descriptions; these, with the user query, form the entire prompt to the 2B model. The model emits one shell command directly—there is no JSON step and no separate template-rendering pass. The command is executed once through a shell (with a per-step timeout and an optional retry), and the pattern's *validators* assert over the result. If the model errors, the command fails to execute, or a validator rejects the outcome, the reflexer does not improvise: it *escalates* to the thinker with a tagged reason. Success is recorded; the loop ends.

A single forward pass is the entire reflexer runtime. There is no tool-use loop, no planning step. When routing is correct and a pattern exists, this constrained executor reaches **100% binding accuracy** on the atom task set (Chapter 5). The point worth stating early is structural rather than numerical: the reflexer succeeds not because the 2B model is strong, but because the pattern has removed almost everything it could fail at.

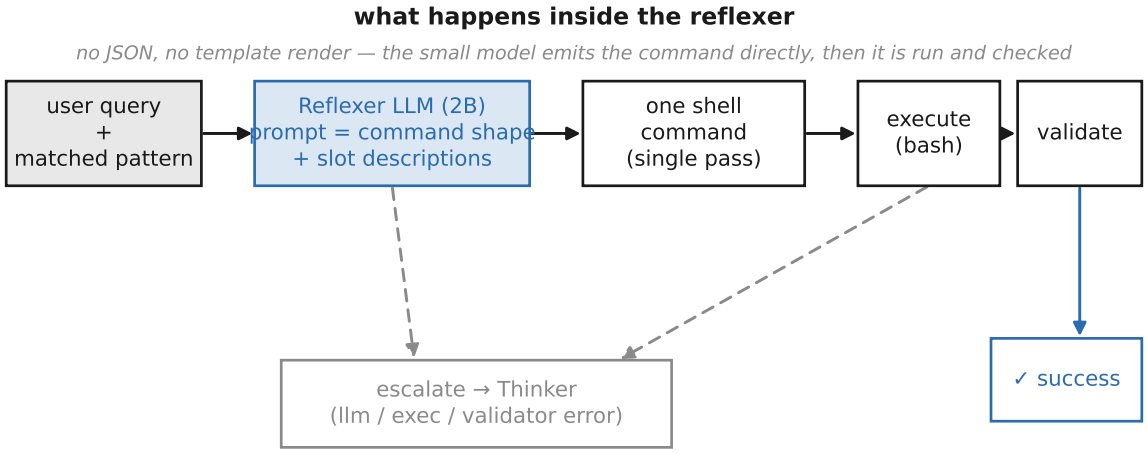

Figure 3.2: Inside the reflexer. The pattern's command shape and slot descriptions prompt the 2B model, which emits a single shell command in one forward pass; the command is executed and validated, and any failure escalates to the thinker rather than being retried by reasoning.

3.3 The thinker: a general agent for the tail

The thinker is a full agent loop in the ReAct style [5]: the model is given the relevant skill documentation and a set of tools (a shell, a file reader), and it reasons, acts, observes, and repeats until it judges the task complete. It is invoked for everything the reflexer cannot serve: multi-intent or compound requests, requests requiring clarification or planning, and any intent for which no pattern has been distilled.

The thinker is where capability lives, but it is also where the architecture pays its costs. An agent loop emits many commands across several turns, consumes substantially more tokens and wall-clock time than a single reflexer call, and at the parameter scale required for reliable reasoning it must run off-device. A recurring theme of the evaluation is that raw thinker capability does not translate cleanly into reliable *binding*: a free-form model, even a large one, tends to substitute its own pretraining priors (real Linux commands, documented backend mechanisms) for the skill's defined vocabulary. The thinker is therefore the right tool for novel problems and the wrong tool for the repeatable head—precisely the asymmetry the two-tier split exploits.

3.4 Patterns: atoms and composites

A **pattern** is the artifact that makes the reflexer possible. It is a small, immutable JSON record that encodes one repeatable intent. Each pattern carries: a trigger (how the router recognizes the intent), a set of typed *slots* (the variable parts to extract from the request), an `args_template` (the exact command shape, with slot placeholders), optional validators (constraints on slot values), and natural-language example queries used by the router's

embedding index.

Patterns come in two kinds. An **atom** encodes a single action—set the screen brightness, start a timer, toggle Wi-Fi—and binds to exactly one command. A **composite** encodes a routine made of several atoms executed in sequence, such as a "good night" action that dims the screen, enables Do Not Disturb, and sets an alarm. Atoms are the unit of reliability: they are stable, reusable, and in our measurements essentially perfectly reflexable. Composites are more demanding, because some require values produced at runtime (a contact name, a URL) rather than known in advance; these remain the harder case for the reflexer and, where they need open-ended slot extraction, are better left to the thinker.

Critically, the `args_template` is a *hard* constraint, not a suggestion. Where the thinker reads skill prose and may reason its way to a different command, the reflexer is handed the command shape directly and only fills the slots. This distinction—soft prose for the thinker versus a hard template for the reflexer—explains much of the reliability gap between the two tiers and is revisited in Chapter 5.

3.5 The skill model: deep modules behind a CLI

A **skill** packages a device capability behind a command-line interface and a documentation file (`SKILL.md`). The CLI is the only surface either tier touches: the reflexer fills a template that calls it, and the thinker reads the `SKILL.md` and invokes it through the shell. Skills emit structured results (a JSON object reporting success, data, and errors), so that outcomes can be checked uniformly regardless of which tier produced the command.

The skill model is an application of Ousterhout's principle of *deep modules*: a simple interface that hides substantial implementation complexity [21]. A well-designed skill exposes a small, intent-shaped vocabulary (`clock timer -duration 10m`) while absorbing the messy device-specific mechanism behind it. This depth is what lets even a small model act competently when it is given the right tool: on a clean, deep settings interface the 2B reflexer succeeds on multi-step tasks that it fails when forced to manipulate the raw state directly (Chapter 5). The corollary, also measured, is that a *leaky* skill—one whose documentation exposes its backend mechanism—degrades the thinker, which reasons toward the familiar mechanism instead of the intended alias. Interface design is therefore not cosmetic; it is a determinant of binding accuracy.

3.6 Distillation: from teacher to a symbolic pattern

Patterns are not written by hand at scale; they are *distilled*. A capable **teacher** model is used once, offline, to convert a skill's documentation and device conventions into the pattern that the cheap on-device **student** (the reflexer) then applies forever. This is the

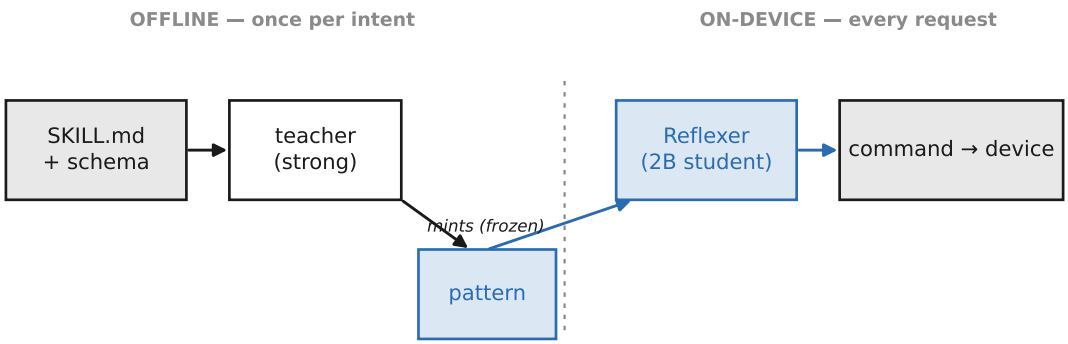

Figure 3.3: Symbolic distillation. A strong teacher reads the skill documentation and schema and mints a frozen JSON pattern once, offline; the cheap 2B student applies that pattern on-device on every subsequent request. The teacher's cost is paid at distillation time, not at inference time.

mechanism that pays for the reflexer's narrowness: the one-time cost of minting a pattern is amortized over every subsequent on-device invocation.

The relationship to classical knowledge distillation is deliberate and worth making precise. In the original formulation, a large teacher network is compressed into the *weights* of a smaller student network [16]. A.L.F.R.E.D. distills along the same teacher→student axis, but into a different target: a teacher model compresses its knowledge of a device's conventions into an *inspectable symbolic artifact*—the pattern—rather than into student parameters. The trade is explicit. Weight distillation yields a general student but requires retraining to extend and is opaque to inspection. Symbolic distillation yields a student that is plug-and-play (adding a capability is adding a JSON file, with no retraining), auditable (the encoded convention can be read and corrected), and composable, at the cost of covering only the intents that have been distilled. For an on-device system that must be extended by skill authors and trusted by users, that trade is the right one.

Two regimes of distillation appear in this work. In **cold-start** distillation the teacher sees only the skill documentation and the API schema, never any task or its expected outcome; it must encode each convention from documentation and prior knowledge alone. In **session-based** (grounded) distillation the teacher additionally observes the real device—reading back actual setting codes before committing an encoding. Cold-start is cheaper and captures well-documented conventions, but it cannot recover genuinely device-specific values that no amount of reasoning can derive; those require grounding. The boundary between what cold-start captures and what only grounding can fix is one of the empirical results of Chapter 5. A strict integrity rule governs both regimes: the teacher must never see the evaluation tasks or their gold outcomes, and a distilled pattern, once minted, is frozen and never edited after observing failures—editing it would leak the benchmark into the artifact under test.

3.7 Design principles

Three principles run through the design and are stated here because they recur as justifications later.

Contract-first. Each benchmark is specified before it is built. The invariants (state isolation, a single success oracle, the metrics) and the oracle itself are written down as a contract, and only then is the mechanism implemented against that contract. This ordering prevents the common failure of letting an evaluation drift to match whatever the system happens to produce.

Success is an end state, not a command. A.L.F.R.E.D. is judged on whether the device reaches the requested state without collateral changes, not on whether it emitted a particular command string. This catches two failures that command matching misses: a syntactically valid command that produces the wrong outcome, and an over-eager agent that achieves the goal but also changes things it was not asked to. The end-state oracle that enforces this is detailed in Chapter 4.

Distillation is the learning phase; the reflex is frozen. All adaptation, iteration, and feedback live in the offline teaching step. The runtime reflex is a single pass over a fixed pattern and never changes during execution. This separation keeps the on-device path simple and predictable however much effort the teacher spends producing the pattern—the analogue of a practiced human reflex, refined slowly through feedback yet executed instantly and without deliberation.

Together these pieces define the system under test: a router that splits traffic, a reflexer that executes distilled patterns in one shot, a thinker that handles the tail, skills that present deep interfaces, and a distillation step that mints the patterns offline. The chapters that follow put numbers to every claim made here—how much model binding really needs, when the abstraction helps, where scale stops helping, and how much the choice of teacher matters.

Chapter 4

Methodology

This chapter specifies how A.L.F.R.E.D. is evaluated, so that every number in Chapter 5 can be read against a fixed and stated procedure. Two methodological commitments do most of the work and are described first: scoring only the execution stage in isolation (*binding-only*, Section 4.2) and scoring success by the resulting device state rather than the command string (the *end-state oracle*, Section 4.3). The benchmarks, the safeguards against runaway agents, and the integrity rules follow.

4.1 Models and infrastructure

Several models appear in the experiments, all run locally. The project began with `ministral-3-3b` as the reflexer and later adopted `qwen-3.5-2b` after the model sweep of Section 5.1; both are used as command-generating reflexers, and `qwen-3.5-2b` additionally serves as the small-model **thinker** baseline. The large **thinker** is `qwen3.6-35b`, a mixture-of-experts reasoning model (35B total, 3B active) used as the capability ceiling for free-form binding and as a local distillation teacher. The cloud distillation teacher is Claude Sonnet 4.6. The Qwen3.5 family at 0.8B and 4B also appears in the reflexer sweep.

Serving stack and quantization. Local models are served with `llama-server` from `llama.cpp` [22] over its OpenAI-compatible HTTP endpoint, and the agent loop is driven by `pi` [23]. **All models use 4-bit (Q4) GGUF quantization**, so that every reported number reflects a model that fits a phone-class memory budget. The two executor models (`qwen-3.5-2b` and `ministral-3-3b`) are served with **identical decoding parameters**; the qwen-3.5-2b invocation is reproduced verbatim in Listing 4.1 and ministral-3-3b uses the same flags. The decisive settings are a low sampling temperature (0.2) for near-deterministic command generation and, crucially, **reasoning disabled** (`enable_thinking=false`)—the reflexer is an executor, not a reasoner, so chain-of-thought is switched off. The 35B is run in its native reasoning mode (thinking enabled),

since it is evaluated precisely as the free-form reasoning ceiling.

Listing 4.1: Reflexer serving configuration (`qwen-3.5-2b`); `ministral-3-3b` uses the same parameters. Q4 quantization, temperature 0.2, reasoning disabled.

```
llama-server Qwen3.5-2B-UD-Q4_K_XL.gguf --alias qwen-3.5-2b
  --host 0.0.0.0 --port 8005 --ctx-size 4072 --n-gpu-layers 999 --jinja
  --threads 16 --temp 0.2 --top-p 0.95 --top-k 20 --min-p 0.00
  --presence_penalty 0.5 --cache-type-k q8_0 --cache-type-v q8_0
  --metrics --slots --flash-attn on --cache-ram 0
  --chat-template-kwargs '{"enable_thinking":false}'
```

Runs are executed *sequentially*: concurrent traffic to the same endpoint was observed to corrupt token accounting and depress accuracy (Section 4.6). Model identities, ports, and per-run parameters are recorded with every report under `benchmark/reports/`.

4.2 Binding-only evaluation

End-to-end accuracy is the product of two stages: *routing* (does the system pick the right pattern or skill?) and *binding* (given the right target, does it emit the right command?). To measure how much model the execution stage needs, routing must be removed as a confound. The binding-only protocol does this by assuming perfect routing: the reflexer is handed the gold pattern for each task, and the thinker is given the documentation for the correct skill. Only binding is then under test. This isolation is what licenses the comparisons in Sections 5.2–5.3; the cost of treating routing separately is revisited honestly in Section 5.8, where it is identified as the real end-to-end bottleneck.

For the binding experiments, success is scored by *token coverage*: a task carries a set of gold tokens that must appear (case-insensitively) in the union of the commands the model emitted. Both tracks read the same task files and use the same checker—the reflexer harness (`alfred/eval/binding_only_eval.py`) and the thinker harness (`src/run-binding-compare.ts`) implement identical coverage semantics—so that the only difference between conditions is the model and whether it received the pattern. Standardizing the checker across both tracks was a deliberate change to make the headline comparison fair: the reflexer is not scored by an easier rule than the thinker.

4.3 The end-state oracle

Token coverage is adequate for binding, where commands are short and singular, but it is the wrong instrument for the settings benchmarks, where a request can be satisfied by more than one command and a syntactically valid command can still produce the wrong outcome. The settings benchmarks (SET-1 and SET-2) therefore use a different, stricter criterion: an **end-state oracle**. Each task runs against a mutable state backend; after the agent finishes, the oracle compares the final state to a gold end-state and passes the

task only if two conditions hold: the requested keys reached their target values, *and* no other key was changed (no collateral edits).

This two-part definition catches two failures that command matching misses. The first is "right command, wrong outcome"—a command that looks correct but leaves the device in the wrong state. The second is over-action—an agent that achieves the goal but also changes settings it was never asked to touch, which a command-matching scorer would happily pass. The no-collateral clause is not hypothetical: in the distillation experiments an over-verbose pattern caused the student to toggle extra radios, and only the end-state oracle detected it (Section 5.6). The oracle is implemented once per benchmark (`SET-1/oracle.py`, `SET-2/oracle.py`) as the single source of truth for success, following the contract-first principle of Section 3.7: the oracle and the task contract were written before the runner.

4.4 Sandbox isolation and the kill policy

Every settings task runs in a fresh sandbox. The backend state is materialized into a temporary file seeded with the task's initial state, the agent acts only on that copy, and the sandbox is destroyed afterward. No task can see another task's changes, and no run touches real device state, the network, or any persistent user data. If an agent corrupts the state file (for example by writing malformed JSON), the oracle reports the corruption rather than crashing, so the failure is recorded as a failure rather than lost.

Agents that reason in a loop must be prevented from running forever. Two mechanisms bound every run. A *step ceiling* caps the number of commands an agent may issue: for atom binding the ceiling is twice the expected number of moves (an atom should need one command, so the cap is two), and for the exploratory settings runs the ceiling is larger to permit genuine discovery. A *wall-clock backstop* aborts a run that exceeds a time budget regardless of step count, which matters for the slow 35B agent. When a ceiling is hit the agent is aborted and scored on whatever state it has produced, so a non-terminating agent counts as a (possibly partial) failure rather than stalling the benchmark. Both bounds, and the commands each agent actually issued, are recorded in the run report for inspection.

4.5 Benchmarks

Three benchmarks are used, chosen to bracket the difficulty of on-device action.

Expansion skills (binding). Seven execution skills—clock, media, focus, device settings, call, app, and browser—with thirty-six atoms and a set of cross-skill composites, each backed by a mock CLI that emits a structured result. The task set provides one to

two queries per skill (the *balanced-10*) and a larger per-skill set (seventy-two atom tasks), with gold tokens for coverage scoring. This benchmark isolates binding across a realistic skill surface and is the basis for Sections 5.2–5.3.

SET-1 (clean settings API). A friendly settings interface (`settings set connectivity.wifi off`) with transparent intent-to-path mapping and validation that rejects bad values. Twenty canonical tasks plus ten harder multi-step tasks, scored by the end-state oracle. SET-1 isolates the value of a deep tool on an easy surface.

SET-2 (realistic Android API). An `adb`-style interface (`settings put <ns> <key> <value>`) over thirty-two settings in three namespaces (`system`, `secure`, `global`), with non-obvious encodings—brightness on 0–255, timeouts in milliseconds, integer enums for Do Not Disturb, ringer, and location, and a string-float font scale—and a permissive `put` that silently accepts wrong inputs, exactly as the real Android tool behaves. Forty-eight tasks, scored by the end-state oracle. SET-2 is the realistic stressor and the setting for the limit-of-scale and distillation results (Sections 5.5–5.6).

4.6 Integrity protocol

Because the central claims rest on a handful of comparisons, the evaluation follows explicit integrity rules, stated here so they can be checked.

1. **No fabricated numbers.** Every figure and table cites a saved run under `benchmark/reports/`; no result is estimated, rounded for effect, or hand-edited. Where two runs of the same condition disagree, both are reported rather than reconciled silently.

2. **No benchmark leakage into distilled artifacts.** In the distillation experiments the teacher is given only the skill documentation and the API schema, never any task or its gold outcome. A pattern, once minted, is *frozen*: it is never edited after observing which tasks it failed, because editing it would leak the benchmark into the artifact under test. The frozen local-teacher patterns are committed as evidence (`SET-2/patterns_distilled_pi.json`).

3. **Isolation.** Runs never touch real user state, never call the device's execution-logging path, and never use the network; each settings task runs in a destroyed-after-use sandbox (Section 4.4).

4. **Sequential execution.** Concurrent runs against the same model endpoint were found to corrupt token accounting and depress accuracy, so the reported runs are executed one at a time.

These rules are the methodological backbone of the thesis: they are what allow a small number of comparisons to carry a strong claim. The next chapter applies the procedure defined here and reports what it found.

Chapter 5

Evaluation

This chapter tests the central claim of the thesis in five steps. We first establish how much model *binding* actually needs, with routing held fixed (Section 5.2). We then explain *why* a small free-form model fails (Section 5.3), and show that the value of the reflexer depends on the difficulty of the device interface (Section 5.4). Section 5.5 reports the result the rest of the thesis turns on: there are task difficulties that model scale provably cannot solve, but a distilled pattern can. Section 5.6 closes the loop by distilling those patterns automatically and measuring how much the choice of teacher matters. The last three sections place the work in context—real-device coverage (Section 5.7), the honest bottleneck (Section 5.8), and the economics (Section 5.9).

Every number below is taken from a saved run under `benchmark/reports/`; the cited filename accompanies each result. Two evaluation disciplines hold throughout, and are detailed in Chapter 4: *binding-only* scoring (routing assumed perfect, so that only the execution stage is under test) and the *end-state oracle* (success is the final device state, not the command string). No number in this chapter is estimated or hand-edited.

5.1 Choosing the reflexer model

Before comparing tiers, the reflexer model itself must be fixed, since the rest of the chapter holds it constant. The work began with `ministral-3-3b`, an instruction-tuned 3B model, which already bound the deterministic slice well; the question was whether a smaller or different model could do as well or better. We swept the Qwen3.5 family (0.8B, 2B, 4B) against `ministral-3-3b` on the 409-atom expansion set, binding-only, each model handed the gold pattern (`binding-only-qwen3.5-*-atoms.txt`, `binding-only-all-ministral-2026-06-16.`

Two findings come out of the sweep. First, `qwen-3.5-2b` is the best reflexer found: perfect binding across all 409 atoms, beating both the smaller 0.8B and the larger ministral-3-3b, at the smallest size that fits a 4 GB device budget—so it is adopted for the rest of the thesis. Second, and more interesting, the small models fail in *model-specific signatures* rather than a single universal weakness. The 0.8B is *semantically* correct but *schema-*

Table 5.1: Reflexer model sweep, 409 expansion atoms, binding-only. The 2B is the smallest model with perfect binding *and* perfect vocabulary compliance.

Model (Q4)	Atom binding	Signature failure mode
qwen-3.5-0.8b	380/409 (92.9%)	violates vocabulary (emits `docker`, not the `dock` alias;
ministral-3-3b	391/409 (95.6%)	prunes static flags (`--skip-download`; youtube
qwen-3.5-2b	**409/409 (100%)**	none
qwen-3.5-4b	100% on captured subset[†]	none observed

[†] A full 409-run for 4B was not persisted (only a docker+service subset, 24/24); it is not reported as a full figure. On the larger 627-task set including core skills, ministral-3-3b scores 601/627 (95.9%).

noncompliant: it emits the real binary name (`docker start`) instead of the skill's defined alias (`dock start`), failing every docker task even at temperature 0.2, because it cannot override a strong lexical prior. The 2B complies perfectly, at parity with 4B. Vocabulary compliance is therefore a **capability threshold between 0.8B and 2B**, not a smooth gradient—scale beyond 2B buys nothing here. The ministral-3-3b signature is different again: it obeys the vocabulary perfectly but prunes semantically-secondary static flags (`--skip-download`), which is why a capable 3B nonetheless drops to 81% on the affected skill. The lesson for the design is that a reflexer must be chosen for *schema obedience*, not raw capability, and that a 2B model already clears that bar.

5.2 The binding spine: distillation substitutes for scale

The first experiment fixes routing and asks a single question: given that the right intent has been identified, how large must a model be to emit the right command? We compare three conditions on the same matched task set (the *balanced-10*: one to two tasks per skill across all seven skills): a 2B reflexer handed the distilled pattern, and a free-form thinker—given only the skill documentation—at 2B and at 35B.

The result, stated plainly: **a 2B reflexer with a distilled pattern matches a 35B free-form thinker (both 100%), while the same 2B without the pattern manages 30%**. The pattern is worth roughly 17× the parameters on this task. The effect persists at scale on the full set: the 2B reflexer binds **72/72 (100%)** atoms across every skill, where the 2B thinker manages only **14/72 (19.4%)** (`binding-only-phase2-qwen2b.txt`, `thinker-binding-phase2-qwen2b.txt`).

Two readings must travel with this headline. First, scale is an *unreliable* lever for binding: the 4B scores below the 2B, partly a context confound but partly a real signal—the 4B emits higher-quality *real* Linux commands (`nmcli radio wifi off`, `playerctl pause`) because it knows the true APIs better, and so obeys the skill's defined surface *less*. Capability to know and obedience to the surface can trade off. Second, this is a

Table 5.2: Binding accuracy on the matched balanced-10 set, with routing held fixed. The 2B reflexer with a pattern equals the 35B free-form thinker; the same 2B without the pattern reaches only 30%.

Condition	Binding	Model size	Median latency	Tokens/task
Reflexer + pattern	**10/10 (100%)**	2B	single call	— (1 call)
Thinker, no pattern	**10/10 (100%)**	35B	6.4 s	∼1.3k
Thinker, no pattern	3/10 (30%)	2B	1.7 s	∼0.7k
Thinker, no pattern	1/10 (10%)	4B†	2.8 s	∼0.7k

† The 4B is configured at context 4000 versus the 35B's 131k, so its thinking crowds the skill document; it is not a clean capability point. Sources: `binding-only-phase2-qwen2b-balanced10.txt`, `thinker-binding-phase2-qwen35b-balanced10.txt`, `thinker-binding-phase2-qwen2b-balanced10.txt`, `thinker-binding-phase2-qwen4b-balanced10.txt`.

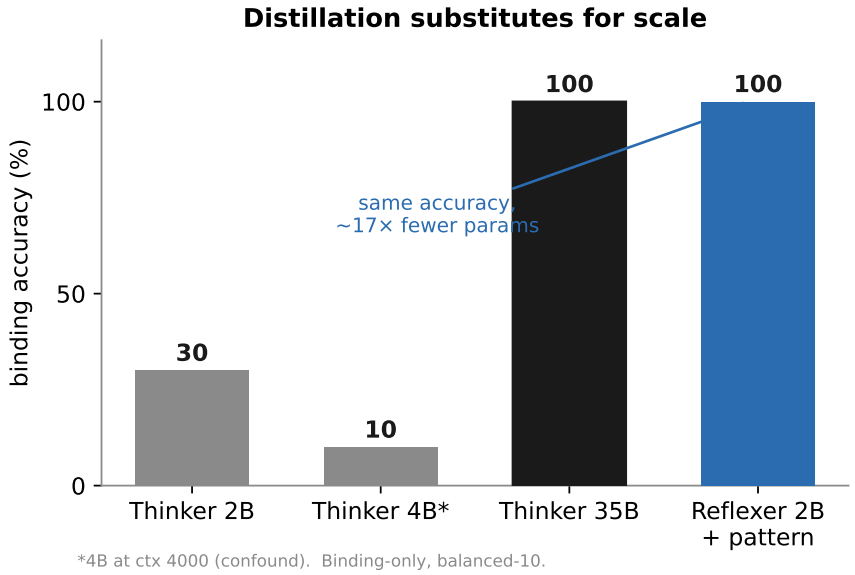

Figure 5.1: Binding accuracy by condition (balanced-10). Scale alone is non-monotonic (4B < 2B < 35B); the pattern takes a 2B to the 35B ceiling.

systems and efficiency result, not a claim that "2B is smarter than 35B." The reflexer is handed the command template and only fills slots, a strictly easier task than free-form generation. The legitimate claim is precise: *on the deterministic slice where intents repeat often enough to distill, a 2B constrained executor matches a 35B free-form thinker's binding accuracy at a fraction of the size, latency, and tokens.*

5.3 Why the small free-form model fails

The 30% of the unaided 2B thinker is not uniform incompetence; it decomposes into two distinct and opposite failure modes, visible when the per-skill numbers are examined (`thinker-binding-phase2-qwen2b.txt`): app 0/6, browser 0/16, call 0/8, clock 1/14,

focus 1/6, media 3/10, and device_settings 9/12.

Under-triggering (no action at all). On all eight `call` tasks the 2B thinker emitted *zero* commands. Short social imperatives— "call mom," "hang up," "answer"—hit the model's conversational prior, and it replies in prose instead of invoking a tool. With no command emitted, the task is an automatic miss. The failure is not wrong output; it is the absence of output.

Wrong vocabulary (real-shell substitution). On `app` tasks the model *does* act, but free-forms from its pretraining prior: `spotify &`, `killall firefox`, `ps aux | grep`, never the skill's defined `app open --name`. Capability is present; obedience to the defined surface is absent.

The diagnostic skill is `clock`, whose documentation deliberately exposes its real backend (`systemd-run`, `systemctl`). Here even the 35B thinker binds only 30% (`thinker-binding-phase2-`
and the 2B only 7% (1/14), while the reflexer binds 100% (14/14). A reasoning model reads the leaked mechanism and prefers the concrete tool it has a strong prior for (`systemctl --user list-timers`) over the abstract `clock` alias it has a weak prior for. For the thinker, the skill document is a *soft* prompt mediated by priors and sampling; for the reflexer, the `args_template` is a *hard* constraint with no prose to reason around. Both failure modes are erased by the pattern, which hard-codes the command shape. This is also a productization argument for distillation: the reflexer is robust to documentation quality, whereas the thinker is not.

5.4 The spectrum: efficiency on clean APIs, accuracy on realistic ones

Whether the reflexer's benefit shows up as *efficiency* or as *accuracy* depends on the interface. Two settings benchmarks bracket the range. SET-1 is a clean, friendly settings API (`settings set connectivity.wifi off`); SET-2 is a realistic Android API (`settings put <ns> <key> <value>`) with three namespaces, non-obvious encodings (brightness 0–255, timeouts in milliseconds, integer enums for Do-Not-Disturb and location, a string-float font scale), and a permissive `put` that silently "succeeds" on wrong inputs—exactly how `adb shell settings` behaves.

On the *clean* API the small model is already a competent executor when it is given a deep tool (Table 5.3). Manual state editing collapses to about a third; the same model driving the deep `settings` script reaches 100%, including two- and three-step compound tasks, because the script rejects wrong guesses and the model recovers. The gain is real but it is *efficiency and competence*—fewer calls, fewer tokens, recovery from error—not

Table 5.3: SET-1 (clean API), qwen-3.5-2b. The deep tool is an efficiency win: +65 percentage points, at 2 calls instead of 9. Sources: `set1-e1e2-qwen2b.txt`, `set1-e2-hard-qwen2b.txt`.

Mode	Oracle pass	Median calls	Median tokens
E1 — manual JSON edit (no script)	∼30–35%	9	3098
E2 — `settings` script	**100%** (20/20; hard 10/10)	**2**	**775**

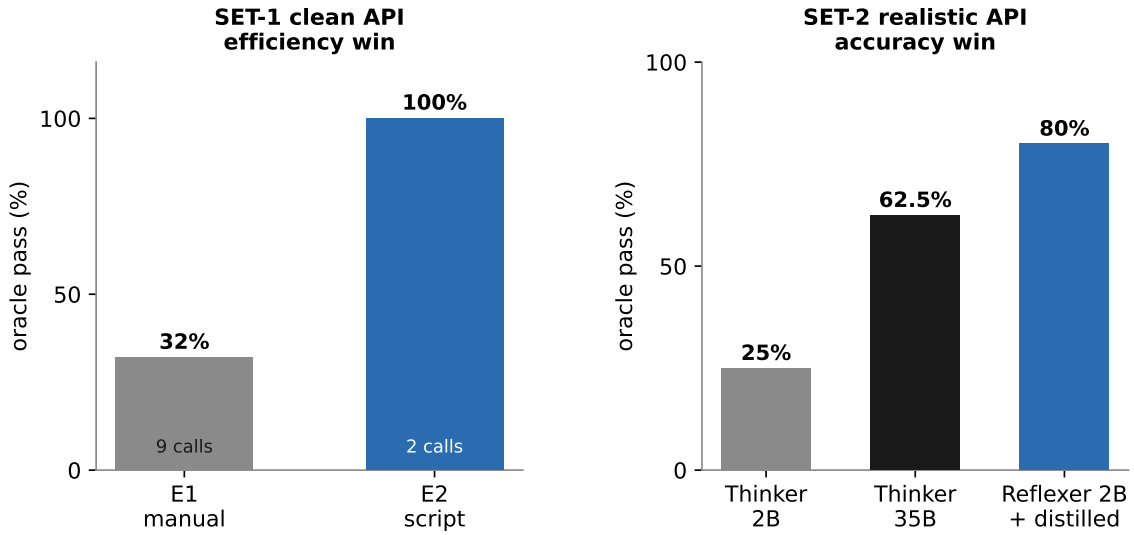

Figure 5.2: The reflexer's benefit scales with discovery difficulty: an efficiency win on a clean API, an accuracy win on a realistic one.

new accuracy the model lacked.

On the *realistic* API the picture inverts. A free-form 2B thinker over the 48 SET-2 tasks scores **12/48 (25%)** (`set2-thinker-qwen2b-48.txt`). Its failures cluster exactly on what cannot be guessed: every brightness percent that must become 0–255, every timeout that must become milliseconds, the integer enums, and the key-name traps (`airplane mode` → `airplane_mode_on`). It passes only guessable booleans and trivial arithmetic. Here the abstraction is no longer about efficiency—a small free-form agent simply *cannot* drive a realistic device-settings API. The reflexer's benefit has shifted from saving calls to supplying accuracy.

5.5 The limit of scale

If a small free-form model fails SET-2, the obvious response is to use a bigger one. This section shows where that response stops working—the result the thesis is built around.

A 35B reasoning thinker (`qwen3.6-35b`) was run on the hardest SET-2 slices: the last three tasks (**3/3**, `set2-thinker-qwen3.6-35b-*.json`) and the five hardest excluding those (**2/5**, `set2-thinker-35b-hard5.txt`), for **5/8 (62.5%)** combined, at roughly nine calls and 17–20 seconds per task, off-device. The 35B clearly beats the 2B—but the

Table 5.4: The 35B thinker's SET-2 failures are conventions, not discovery. These values are not derivable from the API; the model guesses, and is wrong.

Task	Correct value	35B produced
"largest font"	`font_scale "1.30"`	`"1.5"`
"DND total silence"	`zen_mode 2`	`1`
"DND important only"	`zen_mode 1`	`2`

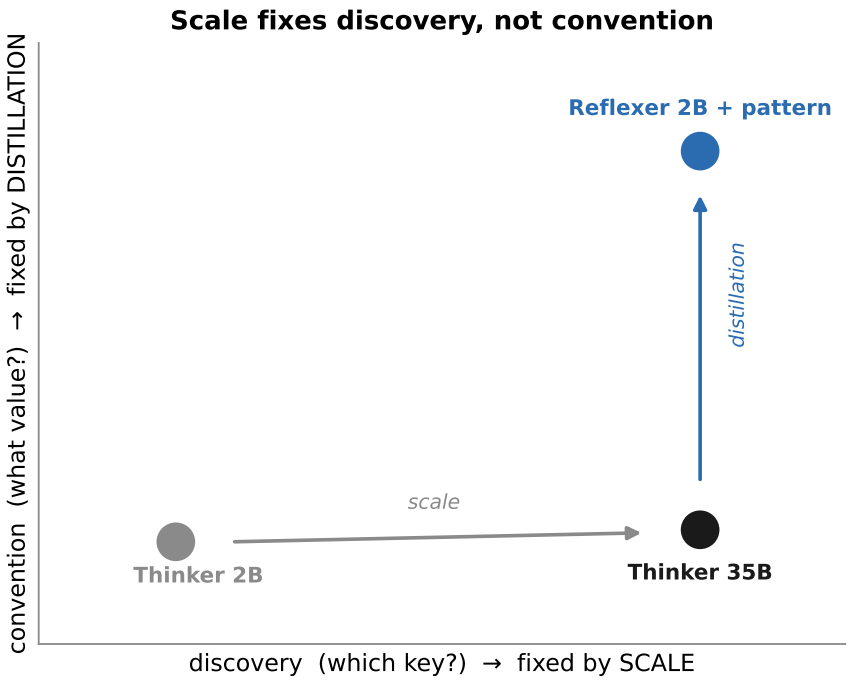

Figure 5.3: Why a distilled pattern beats a 17× larger model on the realistic API: the bottleneck is undiscoverable convention, not reasoning.

manner of its failures is the point. They are not *discovery* failures. The 35B reliably finds the right namespace and key by exploring the API (`list`, `grep`, `get`, then `put`). Its failures are *encoding semantics* (Table 5.4).

This separates two kinds of difficulty by what scale can fix. **Discovery**—*which* namespace and key?—is solvable by exploration, and the 35B does it. **Encoding semantics**—what does the enum value `2` mean, what number is "largest"?—is *not discoverable* from the API at all, and even a 35B guesses and is wrong about half the time. The `zen_mode` error is especially telling: the model makes the *same* mistake every time it is asked, which marks a consistent gap in what the model knows rather than sampling noise. **Scale rescues discovery; scale cannot rescue undiscoverable conventions.** That knowledge exists only in a distilled pattern—which is what the next section supplies.

Table 5.5: Teacher quality propagates to the student: −20 points from the weaker teacher, on the same reflexer and tasks (key-matched mapping, 25 single-setting tasks). Sources: `patterns_distilled.json` (cloud) vs `patterns_distilled_pi.json` (local); `set2-reflexer-qwen-3.5-2b-*`.

Teacher (distiller)	Structure (keys/ns)	Conventions correct	Reflexer pass
Sonnet 4.6 (cloud)	32/32, 0 errors	∼5/7 critical	**20/25 (80%)**
qwen3.6-35b (local)	32/32, 0 errors	∼2/7 critical	15/25 (60%)

5.6 Closing the loop: cold-start distillation and teacher quality

The convention knowledge that no model discovers can be supplied once, offline, by a teacher. We test this in the hardest honest setting: *cold-start* distillation, in which the teacher is given only the skill documentation and the API schema—never any task or its gold outcome—and must mint the patterns from documentation and prior knowledge alone. The 2B reflexer then applies those patterns on the single-setting tasks, scored by the same end-state oracle.

With a capable cloud teacher (Claude Sonnet 4.6), the reflexer rises from the free-form 25% to **19/25 (76%)** on the initial cold-start run, and to **20/25 (80%)** under the key-matched mapping used for the teacher comparison below. The teacher encoded the very conventions the larger in-context thinker got wrong—including `zen_mode = 2` for total silence—and honestly flagged the two it was unsure of (`font_scale` "largest" and `wifi_sleep_policy` "never"), which are genuinely device- or version-specific and not derivable from documentation (`set2-reflexer-qwen-3.5-2b-*.json`). A distilled pattern minted without ever seeing a task therefore beats a 17× larger model that reasons over the live API—because it carries knowledge the API does not expose.

Because the teacher is a swappable component, its quality should propagate to the student, and it does. Repeating the cold-start distillation with a *local* 35B teacher (`qwen3.6-35b`) under an identical prompt, then running the *same* 2B reflexer on each teacher's patterns, gives Table 5.5.

Both teachers are perfect on *structure*—every key, every namespace—so discovery is easy for either. They diverge on *conventions*. The local 35B baked in its own wrong priors: `zen_mode` total-silence as `1` (the same error it made as a thinker, a consistent gap), `location_mode` high-accuracy collapsed to a three-value enum, a wrong brightness worked example, and an over-verbose `airplane` encoding that induced *collateral edits* in the student (the 2B then toggled Wi-Fi and Bluetooth too, failing the no-collateral oracle). It also reported "all verified" while silently wrong, where the cloud teacher flagged its uncertainty. The lesson is operational: cold-start distillation should use the strongest available teacher as a one-time step; a weak local teacher is a weak distiller, and the cheap

reflexer faithfully executes whatever priors it inherits. The two conventions both teachers miss point at the remaining lever—session-based grounding on the real device—which Chapter 6 returns to.

5.7 Coverage: what fraction of real use-cases this addresses

Binding accuracy is meaningful only against a denominator of real demand. Mapping the skill library against the on-device use-cases that Google (Gemini Nano via AICore) and Apple (Apple Intelligence, App Intents) target shows that the reflexer's natural territory is the *deterministic head*: single-command device and media control, smart-home toggles, system and timer operations, and parameterized reads with clear verbs. Apple's App Intents are, in effect, the same construct as a distilled pattern—a pre-declared, parameterized action the on-device model fills rather than invents—which is independent validation of the executioner design. What the reflexer does not cover, by design, escalates to the thinker: multi-intent requests, conversation and planning, and anything outside an installed skill. The qualitative coverage mapping is documented in `docs/research/ondevice_usecases_pixel_apple.md`.

5.8 The honest bottleneck: routing, not binding

This thesis has held routing fixed in order to isolate binding, and binding is effectively solved on the deterministic slice (96–100%). End-to-end coverage, however, is the product of routing and binding, and routing is the limiter. On creative, well-designed skills, routing reaches about 85%; on overlapping skills it falls to 66–68%, so realistic single-intent coverage today is roughly 65–85%, set almost entirely by the router (`routing_ablation_report.md`; `findings_retrieval_and_selection.md`). Decomposed, retrieval is not the bound (the correct candidate is usually in the top-3); *selection* is—the router picks the wrong sibling within or across skills—and selection does not improve with selector scale, only with better prompting. Stating this plainly is part of the contribution: the binding ceiling is reached, and the open problem for a shippable system is the router.

5.9 Economics: when reflexing is worth it

The architecture is ultimately a cost argument, so it should be settled as cost accounting. A pattern is a frozen answer shape: the reflexer never *solves* the task, it fills in a solution found once, which is why a 2B can match a 35B on it. In distillation terms the large model is the *teacher* that mints the pattern once, offline, and the reflexer is the *student* that

Table 5.6: Order-of-magnitude cost model. Break-even is reached at roughly ten invocations, before counting the size, latency, and on-device-feasibility advantages. Source: `findings_thinker_vs_reflexer.md` §9.

	One-time (distill)	Per-invocation (forever)
Reflexer atom	∼5–15k teacher tokens + brief review	∼0.3k tokens, <1 s, fits on-device (2B)
Thinker (big)	0	∼1.3k tokens, ∼6.4 s, off-device (35B)

runs it forever, on-device; the teacher's cost is paid at distillation time, not at inference time, and—unlike a fine-tune—the symbolic pattern stays plug-and-play.

With per-invocation savings of roughly 1k tokens and a one-time cost near 10k tokens to mint an atom, distillation breaks even at about ten invocations (Table 5.6), before the ∼17× per-token size discount, the drop from 6.4 s to under a second, and the fact that the reflexer fits on-device while the 35B does not. For a head intent invoked thousands of times this is a 100–1000× net win; for a tail intent invoked once it never amortizes and should be routed to the thinker. The right design is therefore not "reflexer or bigger model or better router" but all three in their places: distill the high-frequency head, spend the big model as the offline teacher and the tail fallback, and fund the router regardless, because it gates both tiers.

5.10 Summary

The five results compose into one line. A small model cannot drive a realistic device API by reasoning (25%), and scaling the model up to 35B rescues *discovery* but not *convention* (62.5%, with consistent encoding errors). A pattern distilled once by a strong teacher carries exactly that convention, lifting a 2B reflexer to 76–80% at one on-device call—and the quality of that teacher propagates directly to the student (80% vs 60%). On the clean API the same abstraction is instead an efficiency win (+65 points, 2 calls versus 9). Binding, in short, is solved by distillation rather than scale; the remaining open limit is routing. The next chapter draws out what this means for design, for productization, and for the validity of the claims.

Chapter 6

Discussion

6.1 What the results mean

The five results of Chapter 5 compose into a single claim with a sharp boundary. On the repeatable head of on-device tasks, a 2B reflexer with a distilled pattern matches a 17× larger free-form thinker, because the reflexer does not solve the task—it replays a solution found once. The boundary is the distinction the thesis makes precise: model scale buys *discovery*, the search for which namespace and key a request maps to, but it does not buy *convention*, the device-specific meaning of a value. A 35B thinker explores the API and finds the right key, then guesses the encoding and is wrong about half the time, making the *same* mistake every time it is asked (Section 5.5). A distilled pattern carries that convention, so a small model applying it succeeds where the large model reasoning from scratch fails.

This reframes a question usually posed as "how capable a model do we need" into "where does the required knowledge live." For discovery, the knowledge is in the API and a capable model can find it. For convention, the knowledge is in neither the API nor the model's parameters at any scale tested; it exists only in a distilled artifact. Once that is seen, the economics follow directly (Section 5.9): pay a strong teacher once to externalize the convention, and a cheap student applies it forever. The teacher-quality result sharpens the recipe—because the student faithfully executes whatever convention the teacher encoded, a weak teacher bakes in wrong priors, so the one-time distillation step should use the strongest available teacher (Section 5.6).

6.2 Implications for productization

The architecture suggests a clear product shape and an equally clear set of blockers. The defensible first version is a *deterministic-intent accelerator*: the high-frequency, unambiguous head—media and device control, smart-home toggles, timers, system actions—served

on-device by the reflexer at near-100% binding, with the thinker as a safety net for everything else. This is the slice where the value is highest and the risk is lowest, and it coincides with the on-device sweet spot that the Pixel and Apple use-case mapping identifies (Section 5.7).

The blockers are ranked by how much they gate a shippable system. *Routing reliability* is first: end-to-end accuracy is the product of routing and binding, binding is solved, and routing at 66–85% is below what users tolerate for an assistant (Section 5.8); a misroute to the wrong action is the dangerous failure, worse than a refusal. *Cold-start data and authoring cost* is second: each skill needs a pattern set, and patterns must be distilled and, ideally, kept current as device APIs change; the distillation pipeline is what makes this scale. *Composite slot extraction* is third: multi-step routines that need values produced at run time are the high-value automation surface and remain the reflexer's weak point. None of these is a flaw in the executioner idea; they are the engineering distance between a validated mechanism and a product.

6.3 Threats to validity

Three caveats must travel with the headline numbers, and the thesis states them rather than burying them.

First, the comparison is *conditional on a constrained task*. The reflexer is handed the command template and only fills slots, which is strictly easier than free-form generation. The claim is therefore not that a 2B is more capable than a 35B; it is that on the distillable slice a constrained 2B executor reaches the same binding accuracy at a fraction of the cost. Read as a capability claim it would be wrong; read as a systems claim it is what the numbers support.

Second, *coverage is limited to distilled intents*. The 100% binding figure is a ceiling that applies only when a pattern exists and routing has selected it. For intents with no pattern, the system falls back to the thinker, and the reflexer contributes nothing. The approach is a head strategy by construction; its value depends on the workload actually having a repeatable head, which the coverage mapping argues it does.

Third, the strongest results rest on a *small number of comparisons on purpose-built benchmarks* with specific models. SET-2 is realistic but is a model of the Android settings surface, not the full device; the 35B is one large model, not a survey of large models; and two runs of the cold-start reflexer disagreed by one task (19 versus 20 of 25), which the thesis reports rather than reconciles. The qualitative pattern—scale fixes discovery but not convention, distillation fixes both, teacher quality propagates—is robust across the experiments, but the exact percentages should be read as evidence for that pattern rather than as universal constants.

6.4 Limitations

Beyond the threats to validity, four limitations bound the work. The router is evaluated but not solved; it is named as the open bottleneck rather than fixed. Composite slot-filling is identified as weak but not repaired. Both teachers, cloud and local, miss the genuinely undiscoverable conventions (the version- or OEM-specific values), which no cold-start teacher can recover and which require observing the real device. And the reflexer's residual application errors—using an enum label instead of its code, or mis-doing brightness arithmetic—point to a value-transform step that is currently left to the model. Each of these limitations maps onto a concrete next step rather than an open research question, which the next chapter sets out.

Chapter 7

Conclusion and Future Work

7.1 Summary

This thesis asked how much language model reliable on-device action requires, and answered it through A.L.F.R.E.D., a two-tier local-first middleware that routes each request to a tiny reflexer or a heavier thinker. The five research questions are settled by the evaluation.

On scale versus distillation (RQ1), a 2B reflexer with a distilled pattern matches a 35B free-form thinker on binding (both 100% balanced), while the same 2B without a pattern reaches 30%—the pattern is worth roughly 17× the parameters. *On the spectrum* (RQ2), the abstraction is an efficiency win on a clean API (a +65-point oracle gain at two calls instead of nine) and an accuracy win on a realistic one. *On the limit of scale* (RQ3), a 35B thinker rescues discovery but not convention, failing device encodings consistently, while a distilled pattern supplies exactly that convention. *On teacher quality* (RQ4), end-to-end student accuracy tracks the teacher: a cloud teacher's patterns score 80% against a local teacher's 60% on the same reflexer and tasks. *On coverage and routing* (RQ5), the reflexer's territory is the deterministic head that on-device assistants target, and the open bottleneck is routing, not binding.

The contribution, stated once more in plain terms: **on a real device API, scale buys discovery but not undiscoverable convention; distillation buys the convention; and a small reflexer applies it at a fraction of the cost.** The executioner paradigm is validated through the full teacher-to-student loop, and its central mechanism—compressing a teacher's device knowledge into an inspectable symbolic pattern rather than into student weights—is the thesis's main idea.

7.2 Future work

The limitations of Chapter 6 map onto five concrete next steps, ordered by leverage.

Session-based device grounding. Both teachers miss the genuinely undiscoverable conventions because cold-start distillation gives them no way to check a value against reality. Granting the teacher read-only access to the real device—observing actual codes before committing an encoding—would convert those guesses into facts and close the remaining convention errors. The integrity rule carries over: ground on the device or a development split, never on the evaluation gold.

Deterministic value transforms. The reflexer's residual errors are arithmetic and enum-lookup mistakes (percent-to-0–255, label-to-code). Moving these transforms out of the model and into a small deterministic function inside the pattern removes them for both the teacher and the student, and is a bounded engineering change.

The router. The evaluation localizes the open bottleneck precisely: retrieval places the correct candidate in the top-3, but selection picks the wrong sibling, and selection does not improve with selector scale. The next investment is sibling discrimination—better selection prompting or a learned selector over behavioural signals—since the router gates both tiers.

Iterative, grounded distillation. The cold-start patterns can be run, executed on a real device, and re-distilled from the observed failures over two to three rounds, lifting a weak local teacher toward cloud quality without a larger model. This iteration lives entirely in the offline learning phase; the run-time reflex remains a single pass over a frozen pattern, so the simplicity that makes the reflexer deployable is preserved by construction, with a minimality check each round to stop patterns accreting special cases.

Authoring and personalization at scale. Longer-term, mining a user's own repeated trajectories into candidate patterns would extend coverage beyond hand-authored skills, and a privacy-preserving sharing layer would let patterns be exchanged without leaking personal data. A parametric alternative—fine-tuning a very small model on the pattern conventions—is a path worth measuring against the symbolic approach, trading inspectability for a smaller artifact.

7.3 Closing

The shift toward on-device assistants raises a practical question that this thesis takes seriously: not how powerful a model can be, but how little model an action actually needs once its knowledge is placed where it belongs. The answer found here is that a great deal of on-device action is not a reasoning problem at all. It is a knowledge-placement problem—and once the convention is distilled into a pattern, a small model

executing a frozen reflex is enough.

Bibliography

[1] C. S. Sherrington, *The Integrative Action of the Nervous System*. Yale University Press, 1906. Classic formulation of the reflex arc: a stimulus-driven motor response routed without higher cortical deliberation.

[2] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011. Dual-process account: System 1 (fast, automatic) vs System 2 (slow, deliberate); maps to Reflexer vs Thinker.

[3] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. C. Lin, and P. Molchanov, "Small language models are the future of agentic ai," *arXiv preprint arXiv:2506.02153*, 2025. NVIDIA Research position paper; thesis anchor in introduction.

[4] L. E. Erdogan, N. Lee, S. Jha, S. Kim, R. Tabrizi, S. Moon, C. Hooper, G. Anumanchipalli, K. Keutzer, and A. Gholami, "Tinyagent: Function calling at the edge," in *EMNLP 2024 System Demonstrations*, 2024. arXiv:2409.00608; quantitative anchor for SLM tool-use on-device.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629; canonical loop the Thinker tier implements.

[6] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761.

[7] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2305.15334.

[8] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "Routellm: Learning to route llms with preference data," in *Inter-*

national Conference on Learning Representations (ICLR), 2025. arXiv:2406.18665; closest routing predecessor.

[9] J. Dekoninck, M. Fischer, and M. Vechev, "A unified approach to routing and cascading for LLMs," *arXiv preprint arXiv:2410.10347*, 2024. Positions A.L.F.R.E.D.'s three-tier path as a structured cascade.

[10] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2305.16291; original skill-library framing for LLM agents.

[11] R. Fang, X. Wang, Y. Liang, S. Qiao, J. Wu, Z. Xi, N. Zhang, Y. Jiang, P. Xie, and F. Huang, "Memp: Exploring agent procedural memory," *arXiv preprint arXiv:2508.06433*, 2025. Closest "trajectory-to-skill distillation" neighbour; A.L.F.R.E.D. patterns are Memp's script abstractions compiled into deterministic templates.

[12] H. Zhou, Y. Chen, S. Guo, X. Yan, K. H. Lee, Z. Wang, K. Y. Lee, G. Zhang, K. Shao, L. Yang, and J. Wang, "Memento: Fine-tuning LLM agents without fine-tuning LLMs," *arXiv preprint arXiv:2508.16153*, 2025. Foundation of the Memento line; non-parametric CBR over a growing Case Bank. CORRECTED arXiv ID (not 2408.11816).

[13] H. Zhou, S. Guo, A. Liu, Z. Yu, Z. Gong, B. Zhao, Z. Chen, M. Zhang, Y. Chen, J. Li, R. Yang, Q. Liu, X. Yu, J. Zhou, N. Wang, C. Sun, and J. Wang, "Memento-skills: Let agents design agents," *arXiv preprint arXiv:2603.18743*, 2026. NEAREST PUBLISHED NEIGHBOUR. Learned skill-router + Read-Write Reflective Learning over markdown skills. Position A.L.F.R.E.D.'s feature-engineered router and compiled patterns against their neural router and free-form skills.

[14] Y. Wang, X. Chen, X. Jin, M. Wang, and L. Yang, "Openclaw-rl: Train any agent simply by talking," *arXiv preprint arXiv:2603.10165*, 2026. Async on-policy RL alternative to A.L.F.R.E.D.'s non-parametric pattern compilation. Cite once as the "weight-level personalisation" path we explicitly did not take.

[15] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366; name-source for our cheap-path Reflexer (disambiguate explicitly in paper).

[16] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015. Foundational knowledge-distillation paper; A.L.F.R.E.D. distills into a symbolic artifact instead of student weights.

[17] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "Dspy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.03714; programmatic-prompt discipline analogue.

[18] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "Agentbench: Evaluating LLMs as agents," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688; standard agent-evaluation pattern adopted.

[19] O. Yoran, S. J. Amouyal, C. Malaviya, B. Bogin, O. Press, and J. Berant, "Assistantbench: Can web agents solve realistic and time-consuming tasks?," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2407.15711; positions A.L.F.R.E.D.'s 53-task suite in the realistic-assistant benchmark line.

[20] Anthropic, "Equipping agents for the real world with Agent Skills." `https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills`, Oct. 2025. Justifies the SKILL.md design choice; A.L.F.R.E.D. is structurally compatible with the open agentskills.io standard.

[21] J. Ousterhout, *A Philosophy of Software Design*. Yaknyam Press, 2018. Deep-module principle underpinning the skill design.

[22] G. Gerganov and llama.cpp contributors, "llama.cpp: Llm inference in c/c++." `https://github.com/ggml-org/llama.cpp`, 2026. Local OpenAI-compatible serving via llama-server; GGUF Q4 quantization.

[23] M. Zechner, "pi: a minimal agent loop (pi-agent-core)." npm package `@mariozechner/pi-agent-core`, 2026. Agent loop driving the Thinker tier and the SET runners.